Complete Delta Lake Hands-on Lab — Databricks Free Edition

Scenario: Student Performance Management and Retail Sales Optimization Platform: Databricks Free Edition Compute: Serverless Language: PySpark + Spark SQL Duration: 90–120 minutes Prerequisites: A Unity Catalog catalog with permission to create schemas and tables.

This lab follows the sequence in your screenshots, with an additional realistic demonstration of small files, `OPTIMIZE`, and Z-Ordering.

Lab structure

Part	Topic	Business scenario
A	Delta table creation	Store student examination scores
B	Schema enforcement	Reject incompatible score data
C	Schema evolution	Add student country information
D	Time travel	Retrieve earlier versions
E	MERGE / UPSERT	Update and insert student records
F	Small files	Simulate retail transaction batches

G	OPTIMIZE	Compact small Parquet files
H	ZORDER	Improve customer-specific queries
I	VACUUM	Inspect obsolete file cleanup

All exercises can be executed in a single Databricks Python notebook. SQL statements can be executed using `spark.sql()`.

Part A — Create the Delta table

Step 1 — Set up Unity Catalog

catalog = spark.sql("SELECT current_catalog()").first()[0]spark.sql(f"CREATE SCHEMA IF NOT EXISTS `{catalog}`.delta_training")spark.sql(f"USE CATALOG `{catalog}`")spark.sql("USE SCHEMA delta_training")print("Catalog:", catalog)print("Schema: delta_training")

This uses the current catalog. If it is `hive_metastore`, switch to a Unity Catalog catalog where you have write access.

Step 2 — Create a DataFrame

data = [(1, "Alice", 50), (2, "Bob", 60)]df = spark.createDataFrame(data, ["id", "name", "score"])df.printSchema()display(df)

Expected output:

id	name	score
1	Alice	50
2	Bob	60

Step 3 — Create the Delta table

df.write \ .format("delta") \ .mode("overwrite") \ .saveAsTable("emp1")

Verify:

display(spark.sql("SELECT * FROM emp1"))

Inspect table metadata:

display(spark.sql("DESCRIBE DETAIL emp1"))

Expected: The table format is `delta`, and the table contains two records.

Part B — Schema enforcement

Step 4 — Create incompatible data

data2 = [(3, "Charlie", "invalid_score")]df2 = spark.createDataFrame(data2, ["id", "name", "score"])df2.printSchema()

The incoming `score` column is `STRING`, whereas the target `score` column is `BIGINT`.

Step 5 — Attempt an invalid append

try: df2.write \ .format("delta") \ .mode("append") \ .saveAsTable("emp1")except Exception as e: print("Schema enforcement triggered") print(str(e)[:1200])

Expected: Delta rejects the incompatible value or schema.

Business explanation: An upstream application sends an invalid examination score. Delta Lake prevents the incompatible data from being appended.

Verify that the table still has two rows:

display(spark.sql("SELECT * FROM emp1"))

Part C — Schema evolution

Step 6 — Add a new column

df3 = df.withColumn("country", lit("USA"))
display(df3)

Step 7 — Append using mergeSchema

#Step 6 — Add a new column
data4 = [(5, "Messi", 50, "Argentina"), (6, "Ronaldo", 50, "Portugal")]
df4 = spark.createDataFrame(data4, ["id", "name", "score","country"])
display(df4)
#Step 7 — Append using mergeSchema
df4.write.format("delta").mode("append").saveAsTable("ak_catalog.hr_schema.emp1")

#df4.write.format("delta").option("mergeSchema", "true").mode("append").saveAsTable("emp1")

df3.write.format("delta").option("mergeSchema", "true").mode("append").saveAsTable("emp1")

==========

```
#Step 6 – Add a new column with mergeSchema False
data5 = [ (6, "Mbappe", 50, "France",2026)]
df5 = spark.createDataFrame(    data5,    ["id", "name", "score","country","year"])
display(df5)
df5.write.format("delta").option("mergeSchema", "false").mode("append").saveAsTable("emp1")
```

Check the schema:

spark.table("emp1").printSchema()

Check the data:

display(spark.sql("SELECT * FROM emp1"))

Expected:

id	name	score	country
1	Alice	50	NULL
2	Bob	60	NULL
1	Alice	50	USA

2	Bob	60	USA

Notice that the original rows contain `NULL` in the newly introduced column. The two additional rows have `USA`.

This also demonstrates that schema evolution does not automatically deduplicate records.

Part D — Delta time travel

Step 8 — View history

display(spark.sql("DESCRIBE HISTORY emp1"))

You should see at least two versions.

Step 9 — Query version 0

display(spark.sql("""SELECT *FROM emp1 VERSION AS OF 0"""))

Expected:

id	name	score
1	Alice	50
2	Bob	60

Step 10 — Compare with latest version

display(spark.sql("SELECT * FROM emp1"))

This demonstrates that Delta retains earlier logical snapshots, subject to data-file retention.

Part E — MERGE / UPSERT

Step 11 — Prepare CDC data

The examination system receives two changes:

- Alice's score changes from 50 to 55.

- David is a new student with a score of 70.

changes = [(1, "Alice", 55),   (4, "David", 70)]df_changes = spark.createDataFrame(changes,   ["id", "name", "score"])df_changes.createOrReplaceTempView("changes")display(df_changes)

Step 12 — Run MERGE

Because the schema-evolution exercise created duplicate student IDs, first create a clean target table for the MERGE demonstration.

spark.sql("""CREATE OR REPLACE TABLE delta_merge_demoUSING DELTAASSELECT *FROM emp1 VERSION AS OF 0""")

Run the upsert:

spark.sql("""MERGE INTO delta_merge_demo AS baseUSING changes AS srcON base.id = src.idWHEN MATCHED THENUPDATE SET   base.name = src.name,   base.score = src.scoreWHEN NOT MATCHED THENINSERT (id, name, score)VALUES (src.id, src.name, src.score)""")

Validate:

display(spark.sql("""SELECT *FROM delta_merge_demoORDER BY id"""))

Expected:

id	name	score
1	Alice	55
2	Bob	60
4	David	70

This demonstrates both update and insert operations in one atomic transaction.

Part F — Small-files demonstration

This is the most important part for demonstrating `OPTIMIZE` in a live training session.

We will simulate 20 retail ingestion batches. Each batch contains 1,000 records.

Step 13 — Create a retail Delta table

```
spark.sql("CREATE CATALOG IF NOT EXISTS retail_catalog")
spark.sql("USE CATALOG retail_catalog")
spark.sql("CREATE SCHEMA IF NOT EXISTS sales_schema")
spark.sql("USE SCHEMA sales_schema")

spark.sql("""
CREATE OR REPLACE TABLE retail_orders_demo (
    order_id BIGINT,
    customer_id BIGINT,
    product_id BIGINT,
    sales_amount DOUBLE
) USING DELTA
""")

from pyspark.sql import functions as F
for batch in range(20):
    start = batch * 1000
    end = start + 1000
    batch_df = (
        spark.range(start, end)
        .select(
            F.col("id").alias("order_id"),
            ((F.col("id") % 500) + 1).alias("customer_id"),
            ((F.col("id") % 100) + 1).alias("product_id"),
            F.round(F.rand(batch + 1) * 5000, 2).alias("sales_amount")
        )
    )

batch_df.write.format("delta").mode("append").saveAsTable("retail_orders_demo
")

# print("20 ingestion batches completed")
```

Verify the record count:

display(spark.sql("""SELECT COUNT(*) AS total_ordersFROM retail_orders_demo"""))

Expected: 20,000 rows.

Step 15 — Inspect active Parquet files before OPTIMIZE

First inspect Delta metadata:

before = spark.sql("""DESCRIBE DETAIL retail_orders_demo""").select("numFiles", "sizeInBytes").first()print("Active files:", before["numFiles"])print("Size in bytes:", before["sizeInBytes"])

Next, list the physical file paths referenced by the table:

files_before = (spark.table("retail_orders_demo").select(F.col("_metadata.file_path").alias("file_path"), F.col("_metadata.file_size").alias("file_size")).distinct())display(files_before)

You should see Parquet file paths. The number depends on Databricks' automatic write optimizations.

Example: Small files before compaction

part-00

part-01

part-02

part-03

part-04

part-05

part-06

part-07

part-08

part-09

part-10

part-11

Illustration of multiple active Parquet files. Actual count is determined by your table metadata.

Part G — OPTIMIZE

Step 16 — Execute compaction

display(spark.sql("""OPTIMIZE retail_orders_demo"""))

Step 17 — Compare active file counts

after = spark.sql("""DESCRIBE DETAIL retail_orders_demo""").select("numFiles", "sizeInBytes").first()print("Before:", before["numFiles"])print("After :", after["numFiles"])print("Reduction:", before["numFiles"] - after["numFiles"])

Illustrative outcome

20

Before OPTIMIZE

1

After OPTIMIZE
These are illustrative values, not guaranteed results. Serverless automatic compaction can change the outcome.

Step 18 — List files after OPTIMIZE

files_after = (spark.table("retail_orders_demo") .select(F.col("_metadata.file_path").alias("file_path"), F.col("_metadata.file_size").alias("file_size")) .distinct())display(files_after)

Compare the paths and sizes with the earlier listing.

Important: `OPTIMIZE` replaces the active set of eligible small files. The older files may remain in storage until VACUUM, so a physical directory listing and an active-file listing are not necessarily identical.

Part H — Z-Ordering

Step 19 — Create a separate dataset

z_df = (spark.range(500000) .withColumnRenamed("id", "order_id") .withColumn("customer_id", (F.col("order_id") % 10000) + 1) .withColumn("sales_amount", F.round(F.rand(42) * 10000, 2)))z_df.repartition(32) \ .write.format("delta") \ .mode("overwrite") \ .saveAsTable("retail_zorder_demo")

Step 20 — Execute a selective query

display(spark.sql("""SELECT *FROM retail_zorder_demoWHERE customer_id = 501"""))

Inspect the query profile.

Step 21 — Apply Z-Ordering

display(spark.sql("""OPTIMIZE retail_zorder_demoZORDER BY (customer_id)"""))

Step 22 — Run the same query again

display(spark.sql("""SELECT *FROM retail_zorder_demoWHERE customer_id = 501"""))

Compare scan metrics using the query profile.

Z-Ordering can improve data skipping when data spans enough files and the selected column has useful statistics. With 500,000 records, the difference may be small or invisible.

Part I — VACUUM

Step 23 — View table history

display(spark.sql("""DESCRIBE HISTORY retail_orders_demo"""))

Look for `WRITE` and `OPTIMIZE` operations.

Step 24 — Preview obsolete files

display(spark.sql("""VACUUM retail_orders_demo DRY RUN"""))

The command previews files eligible for cleanup without deleting them.

Do not reduce the retention period below the safety threshold for this exercise.

Final validation

Run this in the notebook to review the main tables:

tables = ["emp1", "delta_merge_demo", "retail_orders_demo", "retail_zorder_demo"]for table in tables: print("\nTABLE:", table) display(spark.sql(f"DESCRIBE DETAIL {table}").select("format", "numFiles", "sizeInBytes")) print("Rows:", spark.table(table).count())

Expected learning outcomes

Topic	What trainees should understand
Delta table	Data is stored in Parquet with a Delta transaction log

Schema enforcement	Incompatible data is rejected
Schema evolution	New columns can be added safely
Time travel	Earlier table versions can be queried
MERGE	Existing records are updated and new records inserted
Small files	Repeated ingestion can create file fragmentation
OPTIMIZE	Eligible small files are compacted
ZORDER	Data locality can improve selective queries
VACUUM	Obsolete physical files can be cleaned up after retention

Training recommendation: Use the Alice/Bob dataset for the first five topics because the results are easy to verify manually. Use the retail dataset for OPTIMIZE and Z-Ordering because these operations are meaningful only when enough physical data files exist.

All the examples use managed Unity Catalog Delta tables and avoid `/mnt/...` paths, DBFS mounts, external storage accounts, and dedicated clusters.

Can I use /mnt/path

No. In Databricks Free Edition with Serverless Compute, you should not use `/mnt/path` for Delta table storage.

The `/mnt/` convention is associated with legacy DBFS mounts, which are not supported in Databricks Free Edition's serverless environment.

Supported alternatives

Storage path	Free Edition	Recommendation
`/mnt/lab4/base`	❌	Legacy DBFS mount path
`dbfs:/mnt/lab4/base`	❌	Requires legacy DBFS mounts
`/dbfs/...`	❌	Not suitable for serverless
`/Volumes/catalog/schema/volume/folder`	✅	Best for inspecting Delta files
Unity Catalog managed table	✅	Best for normal Delta operations

Option 1 — Use a Unity Catalog Volume

Create a Volume in a schema where you have permission:

CREATE VOLUME IF NOT EXISTS delta_files;

Write the Delta data:

data = [(1, "Alice", 50), (2, "Bob", 60)]df = spark.createDataFrame(data, ["id", "name", "score"])path = "/Volumes/<catalog>/<schema>/delta_files/base"df.write.format("delta") \ .mode("overwrite") \ .save(path)

Replace `<catalog>` and `<schema>` with your actual names.

To see the physical files:

display(dbutils.fs.ls(path))

You can also navigate to Catalog Explorer → Volumes → delta_files → base.

For the exercise, you can read the Delta dataset directly:

display(spark.read.format("delta").load(path))

Note that using a Volume path for a registered table's storage location can be subject to Unity Catalog restrictions. For a registered production-style table, use a managed table instead.

Option 2 — Use a managed Delta table

```
df.write.format("delta") \    .mode("overwrite") \    .saveAsTable("emp1")
```

Then:

```
SELECT * FROM emp1;

DESCRIBE DETAIL emp1;

OPTIMIZE emp1;

DESCRIBE HISTORY emp1;
```

Databricks Unity Catalog & Delta Lake Optimization — Complete Guide

We'll cover two areas using a Retail E-Commerce Data Engineering Project:

1. Unity Catalog architecture: Catalog, Schema, Table, Volume, Managed (Internal), External, and their real-world use cases.
2. Delta Lake optimization: Small-file compaction, Z-Ordering, Liquid Clustering, Data Skipping, Partitioning, VACUUM, OPTIMIZE, and other performance techniques.

All examples focus on Databricks with Unity Catalog, including what you can practice in Databricks Free Edition.

Part 1 — Unity Catalog Architecture

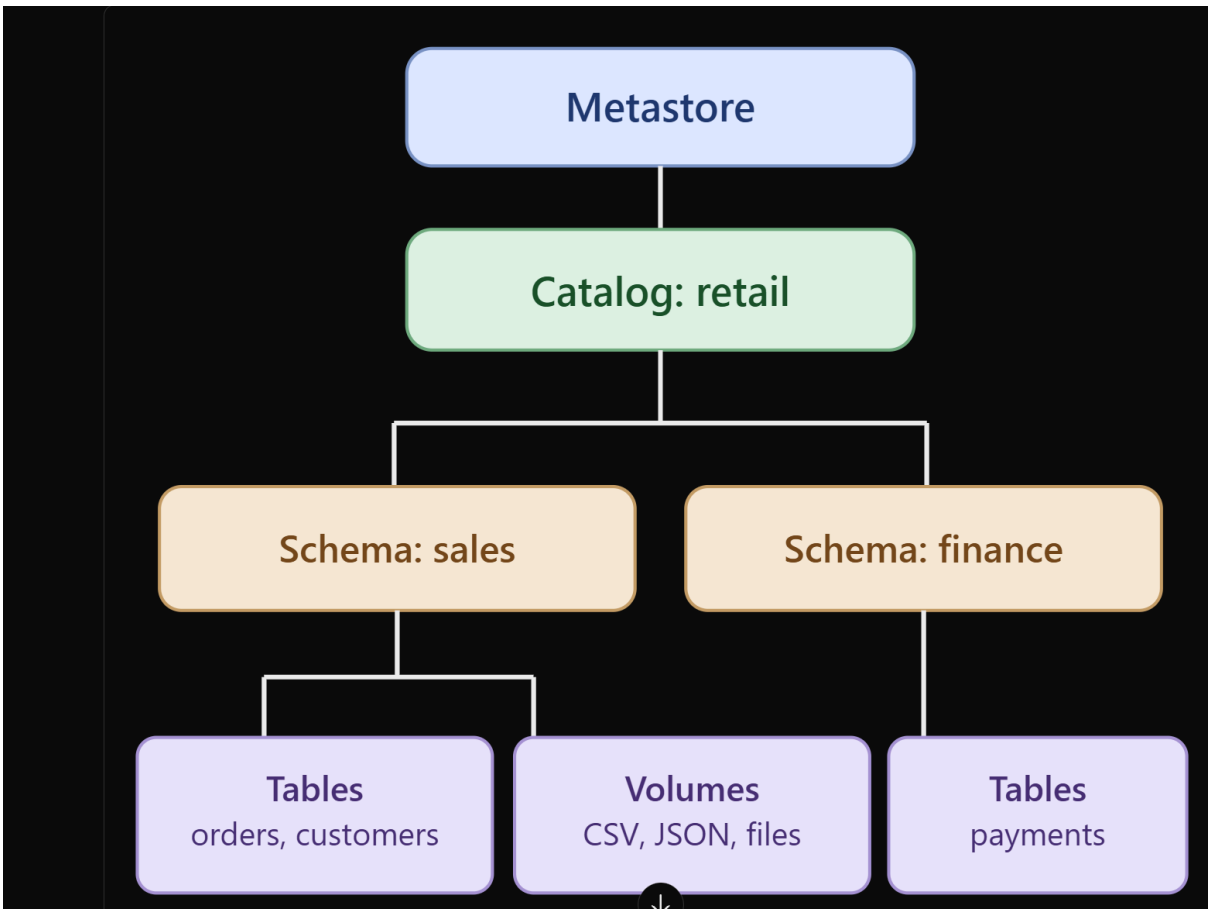
1. What is Unity Catalog?

Unity Catalog is Databricks' centralized data governance system. It manages metadata, permissions, discovery, auditing, and access to data and other governed assets.

In a real-world project, a company may have several business departments, such as Sales, Finance, HR, and Marketing.

Unity Catalog helps organize their data and control which users can access it.

Unity Catalog hierarchy



2. Catalog — Top-level logical container

A Catalog organizes related schemas, tables, views, and volumes.

Example:

retail_catalog
finance_catalog
hr_catalog

When and why do we use a catalog?

Use separate catalogs when you want clear boundaries between environments, departments, or security domains.

For example:

retail_dev
retail_test
retail_prod

A production data engineering team might use `retail_prod` to restrict access to approved users while developers work in `retail_dev`.

CREATE CATALOG IF NOT EXISTS retail_dev;

In Free Edition, use your existing workspace catalog if you do not have permission to create another catalog.

3. Schema — Logical grouping inside a catalog

A Schema (also called a database in Databricks SQL) groups related objects.

Example:

retail_catalog
    sales
    inventory
    customers

Create a schema:

```
CREATE SCHEMA IF NOT EXISTS
retail_dev.sales;
```

When and why?

Use schemas to organize objects by business domain or processing layer.

For example:

```
retail_catalog
    bronze
    silver
    gold
```

- Bronze: Raw ingested records.
- Silver: Cleaned, validated, deduplicated data.
- Gold: Business-ready aggregates and reporting tables.

4. Table — Structured data

A Table stores structured records with defined columns and data types.

For example, a retail orders table:

order_id	customer_id	amount
101	C01	2500
102	C02	3500
103	C01	1800

Create a Delta table:

```
CREATE TABLE retail_dev.sales.orders (
    order_id INT,
    customer_id STRING,
    amount DOUBLE
)
USING DELTA;
```

When and why?

Use tables for structured data that needs SQL querying, joins, aggregations, transactions, schema enforcement, and controlled access.

Delta tables add ACID transactions, history, schema evolution, and optimization capabilities.

5. Volume — Governed file storage

A Volume is a Unity Catalog object used to manage access to files and directories rather than queryable table rows.

Examples include:

- Raw CSV and JSON files.
- Images, PDFs, and text documents.
- Machine learning datasets.
- Files exchanged with external applications.

Create a Volume:

CREATE VOLUME IF NOT EXISTS retail_dev.sales.raw_files;

A file inside it is accessed using:

/Volumes/retail_dev/sales/raw_files/orders.csv

Read the CSV using PySpark:

```
df = (    spark.read    .option("header", "true")
.csv(        "/Volumes/retail_dev/sales/"
"raw_files/orders.csv"    ))
```

Table versus Volume

Feature	Table	Volume
Main purpose	Structured datasets	File-level access
Query using SQL	Yes	Not directly
Schema	Table schema	No row-level schema
Delta transactions	For Delta tables	Not inherently

Access	`catalog.schema.table`	`/Volumes/catalog/schema/volume/`
Typical use	Sales transactions	Source CSV, JSON, PDF

Important: A Volume is not a replacement for a Delta table. Use Volumes to manage files, and tables to manage structured datasets.

6. Managed (Internal) versus External Tables

In Databricks Unity Catalog, the usual terminology is managed table and external table. "Internal table" generally means managed table.

Managed table

Databricks and Unity Catalog manage the table's metadata and storage lifecycle.

```
CREATE TABLE retail_dev.sales.orders_managed
(
    order_id INT,
    amount DOUBLE
)
```

USING DELTA;

You do not specify a `LOCATION`.

When you drop a managed table, its underlying data is subject to Unity Catalog's managed-table deletion lifecycle, including the applicable recovery period.

External table

The data is stored in an explicitly configured cloud storage location, while Unity Catalog manages the table metadata and access governance.

```
CREATE TABLE retail_dev.sales.orders_external (
    order_id INT,
    amount DOUBLE
)
USING DELTA
LOCATION
'abfss://data@storageaccount.dfs.core.windows.net/orders/';
```

This requires the appropriate external location, storage credential, and permissions. It is not a standalone Free Edition example.

Dropping an external table removes its registration but does not delete its underlying cloud data.

Managed versus External

Feature	Managed table	External table
Storage location	Managed by Unity Catalog	Explicit cloud location
Storage ownership	Databricks-managed lifecycle	Organization-controlled
Drop table	Data eventually deleted	Data retained
Governance	Unity Catalog	Unity Catalog
Best for	New Databricks workloads	Existing/shared data lakes

Free Edition	Recommended	Requires supported external storage setup

What about managed and external Volumes?

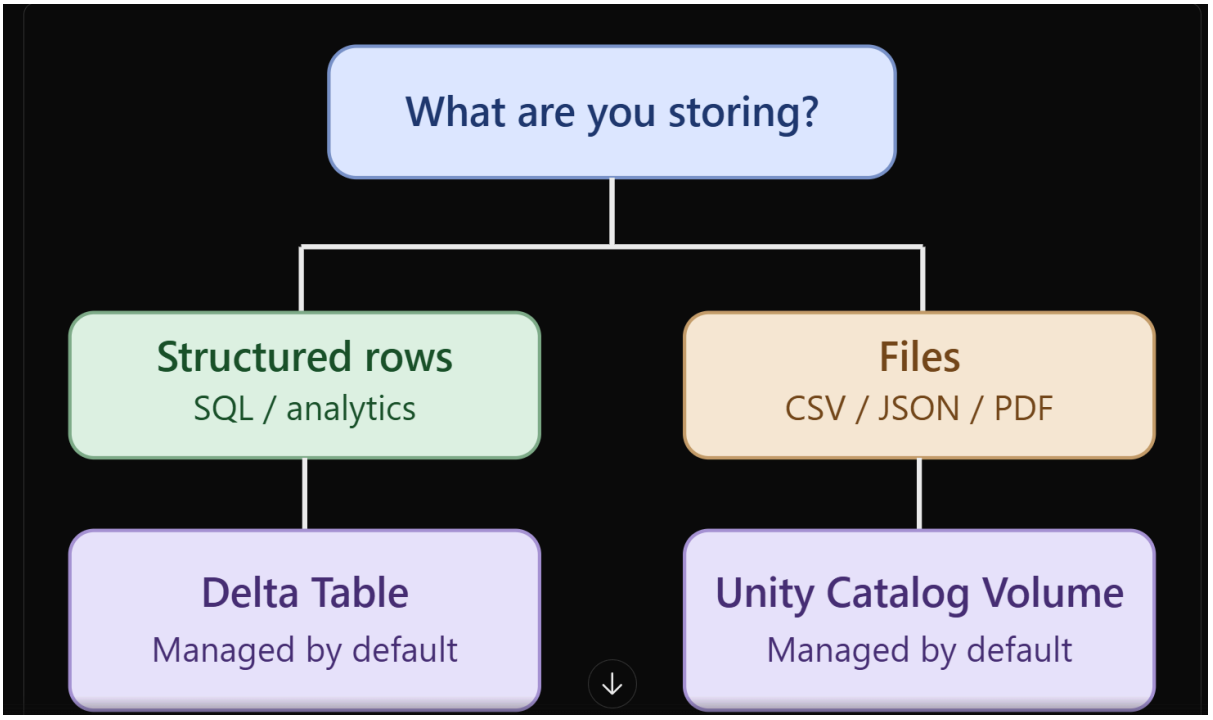
Volumes also come in two forms.

Managed Volume: Unity Catalog manages the storage lifecycle of the files.

External Volume: Unity Catalog governs access to files in an existing cloud storage location.

A real project might use an external Volume to access incoming files in Azure Data Lake Storage, then write processed records into managed Delta tables.

Which should you choose?



For your Databricks Free Edition training, the simplest approach is to use managed Delta tables and a managed Volume for source files.

Part 2 — Delta Lake Optimization

Why is optimization necessary?

Imagine a retail company receiving 1 million orders per day.

Instead of receiving all records in one batch, its ingestion pipeline runs every few minutes, producing many small Parquet files.

The small-files problem



Before optimization

Many small Parquet files

After optimization

Fewer, larger files
Conceptual illustration: compaction reduces file-management overhead. It does not necessarily reduce the total logical data volume.

Delta Lake provides several optimization techniques, each solving a different problem.

1. OPTIMIZE — Small-file compaction

What: Combines eligible small Parquet files into fewer, larger files.

When: After frequent small-batch ingestion, streaming writes, or when the table contains many small files.

Why: Too many small files increase file-opening, metadata-processing, and scan overhead.

OPTIMIZE retail_orders;

Check before and after:

DESCRIBE DETAIL retail_orders;

Compare `numFiles` and `sizeInBytes`.

Real-time example: A retail company ingests 1,000 orders every five minutes. Over time, thousands of small files accumulate. `OPTIMIZE` compacts eligible files to reduce query overhead.

2. Z-Ordering — Improve data locality

What: Organizes related column values near one another within files, improving the effectiveness of data skipping.

When: Existing non-liquid-clustered Delta tables are frequently queried using selective filters on specific columns.

Why: Queries may avoid reading files that cannot contain the requested values.

```
OPTIMIZE retail_orders
ZORDER BY (customer_id);
```

Example:

```
SELECT *
FROM retail_orders
WHERE customer_id = 501;
```

Z-Ordering is particularly useful when the table contains substantial data and the selected columns have file-level statistics.

It does not guarantee faster execution for every query. Avoid Z-Ordering on too many columns, and do not combine it with liquid clustering.

3. Liquid Clustering — Modern data layout optimization

What: Organizes data using clustering keys without requiring traditional partition directories or Z-Ordering.

When: Creating new Delta tables with frequent selective queries, especially when query patterns may change.

Why: Clustering keys can be changed without immediately rewriting all historical data. Subsequent optimization can reorganize files using the updated keys.

```
CREATE TABLE retail_orders_clustered (
    order_id BIGINT,
    customer_id BIGINT,
    order_date DATE,
    amount DOUBLE
)
USING DELTA
CLUSTER BY (customer_id);
```

Optimize:

```
OPTIMIZE retail_orders_clustered;
```

Change the clustering key:

```
ALTER TABLE retail_orders_clustered
CLUSTER BY (order_date);
```

This changes the clustering configuration; it does not immediately reorganize all existing files.

Databricks recommends liquid clustering over Z-Ordering and traditional partitioning for new tables.



Databricks on AWS

4. Partitioning — Partition pruning

What: Physically groups table data into directories based on partition column values.

When: For existing partition-based designs, especially very large tables where queries regularly filter on low- or moderate-cardinality columns.

Why: Spark can skip entire partitions instead of scanning all data.

```
CREATE TABLE sales_partitioned (
    order_id BIGINT,
    order_date DATE,
    region STRING,
    amount DOUBLE
)
USING DELTA
PARTITIONED BY (region);
```

Example:

```
SELECT *
FROM sales_partitioned
```

WHERE region = 'South';

The engine can prune partitions unrelated to `South`.

Avoid partitioning on high-cardinality columns such as unique order IDs. That can create excessive directories and small files.

For new Databricks tables, liquid clustering is generally preferred.

5. Data Skipping — Avoid reading unnecessary files

What: Delta stores file-level statistics, such as minimum values, maximum values, null counts, and record counts.

When: Queries use filters on columns with collected statistics.

Why: The query engine can eliminate irrelevant files before scanning them.

Example:

```
SELECT *
FROM retail_orders
WHERE amount > 5000;
```

If a file's maximum `amount` is 3,000, the engine knows it cannot contain matching rows.

Data skipping works automatically when suitable statistics are available. You do not run a separate `DATA SKIPPING` command.

Z-Ordering and liquid clustering improve file layout so data skipping can be more effective.



Databricks on AWS

6. Optimized Writes — Reduce small files during ingestion

What: Databricks adjusts the output layout while writing data.

When: Ingestion pipelines repeatedly append data or perform `MERGE` operations.

Why: It reduces the creation of excessively small files.

For supported Delta tables and operations, optimized writes are automatically enabled.

For workloads where the setting is applicable and configurable:

spark.conf.set("spark.databricks.delta.optimizeWrite.enabled", "true")

This is a legacy/session-level tuning control, not a setting trainees generally need to change in Free Edition.

Optimized writes occur during writing. `OPTIMIZE` is a separate operation that can reorganize existing files.

7. Auto Compaction — Automatic file consolidation

What: Automatically compacts small files after supported write operations.

When: Tables receive frequent writes and file counts grow.

Why: Reduces the need for frequent manual compaction.

Example of the relevant setting in environments that support configuring it:

```
spark.conf.set(
"spark.databricks.delta.autoCompact.enabled",
"true")
```

In modern Databricks environments, automatic optimization behavior depends on the runtime, table type, and workload. Avoid forcing legacy settings unless there is a specific reason.

Difference: Optimized writes attempt to produce better-sized files during a write; auto compaction consolidates eligible files after writes.

8. Predictive Optimization — Automated table maintenance

What: Databricks automatically decides when to perform maintenance operations such as `OPTIMIZE`, `VACUUM`, and statistics collection on eligible Unity Catalog managed tables.

When: Production environments contain many managed tables and manual maintenance becomes difficult.

Why: Reduces administrative effort and adapts maintenance to table usage.

Example configuration, where supported and authorized:

```
ALTER TABLE retail_orders
ENABLE PREDICTIVE OPTIMIZATION;
```

Predictive optimization can incur compute costs and is subject to platform and account support. It should not be assumed available for every Free Edition workspace.

Databricks recommends predictive optimization for managed tables.



Databricks on AWS

9. VACUUM — Remove obsolete files

What: Deletes physical data files that are no longer required by the current Delta table and have exceeded the applicable retention threshold.

When: After repeated updates, deletes, merges, and optimization operations have accumulated obsolete files.

Why: Reduces storage consumption.

Preview:

```
VACUUM retail_orders DRY RUN;
```

Execute with the default retention:

VACUUM retail_orders;

Important: `VACUUM` is a storage-cleanup operation, not primarily a query-performance optimization. Removing old files can make older Delta versions unreadable through time travel.

Do not use zero-hour retention for training or production tables.

10. Deletion Vectors — Efficient row-level changes

What: Record deleted or modified rows without necessarily rewriting entire Parquet files immediately.

When: Tables experience frequent `DELETE`, `UPDATE`, or `MERGE` operations.

Why: Rewriting large Parquet files for a small number of row changes can be expensive.

For example:

```
DELETE FROM retail_orders
WHERE order_id = 101;
```

With deletion vectors enabled and supported, Delta can logically mark the affected row instead of immediately rewriting the entire file.

A later file rewrite or maintenance operation can physically apply those changes.



Databricks on Google Cloud

11. Statistics and ANALYZE TABLE

What: Collects optimizer statistics that help the query planner make better execution decisions.

When: Data volumes or distributions change significantly, particularly for complex joins and aggregations.

Why: Accurate statistics help the optimizer choose efficient query plans.

```
ANALYZE TABLE retail_orders
COMPUTE STATISTICS;
```

This computes table statistics. Delta file-level data-skipping statistics are a related but distinct mechanism.

12. Query Optimization — Photon, caching, and AQE

These are execution-engine optimizations rather than physical Delta file-maintenance commands.

Technique	What it does	When to use
Photon	Vectorized query execution engine	SQL analytics, joins, aggregations
Adaptive Query Execution (AQE)	Adjusts query plans using runtime statistics	Joins and shuffles
Data caching	Reuses previously read data where supported	Repeated queries
Broadcast joins	Sends small lookup tables to workers	Large fact table joined with small dimension

Column pruning	Reads only necessary columns	Wide tables
Predicate pushdown	Pushes filters toward the scan	Selective queries

Example of writing an efficient query:

```
SELECT customer_id, SUM(amount)
FROM retail_orders
WHERE order_date >= DATE '2026-01-01'
GROUP BY customer_id;
```

This is generally preferable to reading every column and filtering only after loading the entire dataset into application code.

Part 3 — Which optimization should you choose?

Situation in a real project	Recommended technique	Why

Thousands of small files	`OPTIMIZE`	File compaction
Existing table frequently filtered by customer ID	Z-Ordering	Improves data skipping
New large analytical table	Liquid clustering	Flexible physical data layout
Large legacy partitioned table	Partition pruning	Avoids irrelevant partitions
Repeated small ingestion batches	Optimized writes / auto compaction	Prevents file fragmentation
Many managed production tables	Predictive optimization	Automated maintenance

Many updates and deletes	Deletion vectors	Reduces immediate file rewrites
Obsolete files consume storage	`VACUUM`	Reclaims storage
Complex queries with large joins	Statistics / AQE / Photon	Improves execution efficiency

Part 4 — What can you demonstrate in Free Edition?

Feature	Practical recommendation
Catalog, schema, managed table	Hands-on
Managed Volume	Hands-on
Delta schema enforcement/evolution	Hands-on

OPTIMIZE	Hands-on
ZORDER	Hands-on on a non-clustered table
Liquid clustering	Try with a supported Delta table
Partitioning	Hands-on
Data skipping	Inspect query profiles
VACUUM DRY RUN	Hands-on
External tables/volumes	Explain architecture; needs configured external storage
Predictive optimization	Explain concept; Premium-plan feature

Feature availability and automatic optimization behavior can vary by workspace and compute configuration.

Recommended teaching sequence

For graduate trainees, I recommend a progression from storage concepts to performance tuning:

1. Unity Catalog fundamentals: Catalog → Schema → Managed Table → Volume, followed by managed versus external storage.
2. Delta fundamentals: Parquet files, `_delta_log`, schema enforcement, schema evolution, MERGE, and time travel.
3. Physical optimization: Create 20 small ingestion batches, inspect active files, run `OPTIMIZE`, and compare results.
4. Query optimization: Demonstrate data skipping, Z-Ordering, partition pruning, and liquid clustering on separate tables.
5. Production maintenance: Explain auto compaction, predictive optimization, statistics, deletion vectors, and VACUUM.

The key architectural distinction for trainees is:

Unity Catalog governs the data. Delta Lake provides transactional storage and data-management capabilities. OPTIMIZE, Z-Ordering, and liquid clustering improve how Delta data is physically organized.

One important correction for your earlier Volume-based exercise: Delta files can be written to and read from a Volume path, but Unity Catalog does not allow registering a table whose storage directory is inside a Volume. For SQL table demonstrations, use managed tables; for file-browser demonstrations, use a separate Volume path.



Databricks Documentation
+1